

Project 1: Inventory Management System

Complete Code Explanation & Interview Preparation Guide

C++ | STL | Binary Search | File I/O | OOP | DSA

This document covers the complete line-by-line explanation of your Inventory Management System, including DSA concepts, OOP principles, and interview talking points for each section.

Section 1: Headers & Struct

Code:

```
#include <bits/stdc++.h>
#include <fstream>
#include <sstream>
using namespace std;

struct Product {
    int id;
    string name;
    string category;
    double price;
    int quantity;
    string supplierName;
};
```

Word by Word Explanation:

#include <bits/stdc++.h> — Shortcut header that includes ALL standard C++ libraries at once — vectors, strings, algorithms, everything. Fine for projects and competitive programming.

#include <fstream> — fstream = file stream. Gives you tools to read from and write to files. Without this you cannot touch any file on your computer from C++.

#include <sstream> — sstream = string stream. Lets you treat a string like a stream — extract words/values from it one by one. Used to parse CSV lines.

using namespace std; — namespace is a container that groups related code to avoid name conflicts. std is the standard namespace where cout, cin, vector, string etc live. This line lets you write cout instead of std::cout every time.

struct Product — Structure — an OOP concept to group related variables of different types under one name. Think of it as a blueprint — every product will have these 6 fields.

int id — Integer to uniquely identify each product.

double price — Decimal number for price. double gives more precision than float — stores more decimal places accurately.

struct vs class — In struct everything is public by default. In class everything is private by default. We use struct here because we just need data grouping, no access restrictions.

OOP Concept: Encapsulation — keeping related data bundled in one unit (Product struct). **Interview line:** 'I used a struct to model a Product entity, encapsulating all its attributes together. This follows the OOP principle of encapsulation.'

Section 2: File Persistence (fstream + CSV)

Save to File:

```
void saveToFile(const vector<Product>& inventory) {
    ofstream file("inventory.csv");
    for (const auto& p : inventory) {
        file << p.id << "," << p.name << "," << p.category << ","
            << p.price << "," << p.quantity << "," << p.supplierName << "\n";
    }
    file.close();
}
```

const vector<Product>& inventory — Reference (&) means we point to original vector, not copy it — saves memory. const means this function promises NOT to modify inventory, just read it. This is const correctness.

ofstream — Output file stream — used to WRITE to files. ofstream file('inventory.csv') opens the file for writing. Creates it if it doesn't exist.

for (const auto& p : inventory) — Range based for loop — modern C++ way to iterate over all elements. auto = compiler figures out type automatically (Product here). &p; = reference to each product, avoids copying.

file << p.id << ',' — Insertion operator pushes data into the file stream. We separate each field with comma — that's what makes it a CSV (Comma Separated Values). \n = newline, each product on its own line.

file.close() — Always close file after writing. Flushes the buffer and releases the file. Forgetting this can cause data loss.

Load from File:

```
void loadFromFile(vector<Product>& inventory) {
    ifstream file("inventory.csv");
    if (!file.is_open()) return;

    string line;
    while (getline(file, line)) {
        stringstream ss(line);
        string token;
        Product p;
        getline(ss, token, ','); p.id = stoi(token);
        getline(ss, p.name, ',');
        getline(ss, p.category, ',');
        getline(ss, token, ','); p.price = stod(token);
        getline(ss, token, ','); p.quantity = stoi(token);
        getline(ss, p.supplierName, ',');
        inventory.push_back(p);
    }
    file.close();
}
```

ifstream — Input file stream — used to READ from files. Opposite of ofstream.

if (!file.is_open()) return — is_open() checks if file opened successfully. != NOT. If file did NOT open, return immediately. Handles case where inventory.csv doesn't exist yet (first run).

while (getline(file, line)) — getline reads one full line from file into 'line'. while keeps reading until no more lines. Each iteration = one product.

stringstream ss(line) — Wraps the CSV line like '1,Apple,Fruit,10.5,100,SupplierA' in a stringstream so we can extract fields one by one using getline with comma delimiter.

getline(ss, token, ',') — Reads from stringstream until it hits a comma. First call gets '1', second gets 'Apple', etc. The comma is the delimiter.

stoi(token) — String to Integer conversion. CSV stores everything as text, so we convert back to int. stod = string to double.

inventory.push_back(p) — After parsing all fields, adds the complete product to the vector. push_back appends to end of dynamic array.

Interview line: 'I implemented file persistence using fstream. On startup the program loads existing data from a CSV file using ifstream and stringstream for parsing. On every write operation it saves back using ofstream. This ensures data is retained across sessions.'

Section 3: Add Product

```
void addProduct(vector<Product>& inventory) {
    Product p;
    cout << "\n--- ADD NEW PRODUCT ---\n";
    cout << "Enter Product ID: "; cin >> p.id;
    cin.ignore();
    cout << "Enter Product Name: "; getline(cin, p.name);
    cout << "Enter Category: "; getline(cin, p.category);
    cout << "Enter Price: "; cin >> p.price;
    cout << "Enter Quantity: "; cin >> p.quantity;
    cin.ignore();
    cout << "Enter Supplier Name: "; getline(cin, p.supplierName);
    inventory.push_back(p);
    saveToFile(inventory);
    cout << "Product added and saved successfully!\n";
}
```

vector<Product>& inventory — No const this time — we WILL modify the vector by adding a product. Reference & so changes persist in main.

Product p — Creates an empty Product struct. All fields uninitialized — we fill them one by one from user input.

cout — Character output stream. Prints text to terminal. << pushes text into output stream. \n = newline.

cin >> p.id — Character input stream. >> extraction operator reads from keyboard into variable. cin stops reading at whitespace (space, newline, tab).

cin.ignore() — CRITICAL — when you type 5 and press Enter, cin reads 5 but leaves \n in the buffer. Next getline would read that \n and get empty string. cin.ignore() discards that leftover newline. Forgetting this is a very common bug.

getline(cin, p.name) — Reads entire line including spaces into p.name. Unlike cin which stops at space, getline reads until newline. Needed for names like 'Blue Denim Fabric' with spaces.

saveToFile(inventory) — Called immediately after adding — persists data right away, not lost if program crashes.

OOP Concepts: Abstraction — addProduct hides complexity behind one simple call. Separation of concerns — add, save, display are separate functions each doing one job.

Interview Q: 'Why getline instead of cin for name?' — 'cin stops at whitespace so names with spaces would be read incorrectly. getline reads the entire line.'

Interview Q: 'What is cin.ignore()?' — 'After cin reads a number, it leaves \n in buffer. getline would read that empty newline. cin.ignore() discards it.'

Section 4: Display Inventory

```
void displayInventory(const vector<Product>& inventory) {
    if (inventory.empty()) {
        cout << "No products in inventory!\n"; return;
    }
    cout << left << setw(5) << "ID" << setw(15) << "Name"
        << setw(15) << "Category" << setw(10) << "Price"
        << setw(8) << "Qty" << "Status\n";

    for (const auto& p : inventory) {
        cout << left << setw(5) << p.id << setw(15) << p.name ...
        if (p.quantity == 0) cout << "Out of Stock\n";
        else if (p.quantity <= 5) cout << "Low Stock\n";
        else cout << "In Stock\n";
    }
}
```

inventory.empty() — Built-in vector function. Returns true if vector has zero elements. Good practice — always handle edge cases first.

cout << left — left is a format manipulator from <iomanip>. Aligns text to the LEFT within the column width.

setw(5) — set width — reserves exactly 5 characters of space for the next output. Makes columns align neatly regardless of content length. Only applies to NEXT output, then resets.

if (p.quantity == 0) — == is equality comparison, NOT assignment (=). Checks if completely out of stock.

else if (p.quantity <= 5) — <= is less than or equal to. If not zero but 5 or less — low stock warning.

if-else ladder — Checks conditions top to bottom, first match wins. Classic pattern for mutually exclusive conditions.

DSA: Vector traversal — $O(n)$ time complexity. Edge case handling (empty inventory).

OOP: const correctness — displayInventory promises not to modify data. Single responsibility — only displays.

Interview Q: 'Time complexity of display?' — ' $O(n)$ — we traverse every product once.'

Interview Q: 'What happens if inventory is empty?' — 'I handle that edge case by checking inventory.empty() at the start and returning early with a message.'

Section 5: Binary Search

```
void sortInventory(vector<Product>& inventory) {
    sort(inventory.begin(), inventory.end(),
        [](const Product& a, const Product& b) {
            return a.id < b.id;
        });
}

void searchProduct(const vector<Product>& inventory) {
    int targetId; cin >> targetId;
    int low = 0, high = inventory.size() - 1;
    bool found = false;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (inventory[mid].id == targetId) { found = true; break; }
        if (inventory[mid].id < targetId) low = mid + 1;
        else high = mid - 1;
    }
    if (!found) cout << "Product not found.\n";
}
```

sort(inventory.begin(), inventory.end(), comparator) — STL sort algorithm. $O(n \log n)$ time. `begin()` = iterator to first element. `end()` = iterator to one past last. Together = sort entire vector.

Lambda `[](const Product& a, const Product& b) { return a.id < b.id; }` — Anonymous inline function. `[]` = capture clause (empty = no outside variables used). Returns true if a should come before b. Result: sorts by ID ascending. Binary search ONLY works on sorted data.

int low = 0, high = inventory.size() - 1 — Search boundaries. `size()-1` because arrays are zero indexed — last element is at index `size-1`.

bool found = false — Boolean flag to track if product was found. Used at end to print 'not found' message.

while (low <= high) — Keeps searching while valid range exists. When `low > high`, search space exhausted — product doesn't exist.

int mid = low + (high - low) / 2 — CLASSIC INTERVIEW POINT — Why not $(low+high)/2$? If both are large integers, their sum can overflow `INT_MAX`. `low + (high-low)/2` gives same result but is overflow safe. Always mention this.

if (inventory[mid].id < targetId) low = mid + 1 — Middle is smaller than target — target must be in RIGHT half. Move left boundary right.

else high = mid - 1 — Middle is larger than target — target must be in LEFT half. Move right boundary left.

DSA Concepts: Binary Search — $O(\log n)$. For 1000 products, max 10 comparisons vs 1000 for linear search. STL Sort — $O(n \log n)$ using Introsort internally (hybrid of quicksort, heapsort, insertion sort).

Interview Q: 'Why binary over linear search?' — 'Binary is $O(\log n)$ vs $O(n)$. 1000 products needs max 10 comparisons.'

Interview Q: 'Why sort before binary search?' — 'Binary search relies on sorted order to eliminate half the search space each step.'

Interview Q: 'Why $\text{low} + (\text{high} - \text{low}) / 2$?' — 'To avoid integer overflow when low and high are both close to INT_MAX.'

Interview Q: 'What is a lambda?' — 'An anonymous inline function. I used it as a comparator to sort products by ID.'

Section 6: Main Menu

```
int main() {
    vector<Product> inventory;
    loadFromFile(inventory);
    int choice = 0;

    while (choice != 5) {
        cout << "1. Add\n2. Display\n3. Search\n4. Delete\n5. Exit\n";
        cin >> choice;
        switch (choice) {
            case 1: addProduct(inventory); break;
            case 2: displayInventory(inventory); break;
            case 3: sortInventory(inventory); searchProduct(inventory); break;
            case 4: deleteProduct(inventory); break;
            case 5: cout << "Goodbye!\n"; break;
            default: cout << "Invalid option!\n"; break;
        }
    }
    return 0;
}
```

int main() — Entry point of every C++ program. Execution always starts here. Returns int — 0 means success.

loadFromFile(inventory) — First thing on startup — loads previously saved products. If file doesn't exist, inventory stays empty.

int choice = 0 — Always initialise variables in C++. Uninitialized variables have garbage values — undefined behavior.

while (choice != 5) — Application loop / event loop. Keeps program running until user exits. != means not equal to. Every menu-driven program uses this pattern.

switch (choice) — More efficient than if-else for menu — jumps directly to matching case in $O(1)$. if-else checks top to bottom in $O(n)$.

break — CRITICAL — exits switch block after each case. Without break, execution falls through to next case — very common bug.

case 3: sortInventory then searchProduct — We sort BEFORE every search to handle newly added products. Binary search needs sorted data.

default: — Handles invalid input like 6, 7, letters. Always include default in switch — good practice.

return 0 — Tells OS program finished successfully. 0 = success by convention.

OOP Concepts: Modularity — main is clean and simple, complexity hidden in other functions.

Abstraction — main doesn't know HOW addProduct works, just calls it. Single responsibility — main only handles menu flow.

Interview Q: 'Why switch over if-else?' — 'Switch jumps to matching case in $O(1)$. More efficient and readable for menu programs.'

Interview Q: 'What happens if you forget break?' — 'Execution falls through to next case and runs it too'

even if it doesn't match.'

Interview Q: 'Why sort before every search?' — 'New products may have been added since last sort. Sorting ensures binary search always works correctly.'

Quick Revision Reference

DSA Concepts Covered:

- Binary Search — $O(\log n)$, works only on sorted data, overflow-safe mid formula
- STL Sort — $O(n \log n)$, uses Introsort internally, lambda comparator
- Dynamic Arrays (vector) — `push_back` $O(1)$ amortized, `size()`, `empty()`, `begin()`, `end()`
- Range-based for loop — modern C++ iteration
- Edge case handling — always check empty before operations

OOP Concepts Covered:

- Encapsulation — Product struct bundles related data
- Abstraction — complex logic hidden behind simple function calls
- Const correctness — `const&` for read-only parameters
- Single Responsibility Principle — each function does exactly one job
- Separation of Concerns — `add`, `save`, `display`, `search` are all separate
- Modularity — main function is clean, complexity in helper functions

Key Interview Points to Remember:

- Binary search is $O(\log n)$ vs linear $O(n)$ — for 1000 items, 10 vs 1000 comparisons
- `mid = low + (high-low)/2` to avoid integer overflow — always mention this
- `cin.ignore()` after `cin>>` before `getline` — prevents empty string bug
- `getline` for names with spaces, `cin>>` for numbers
- Always sort before binary search — binary search needs sorted data
- `file.close()` after every file operation — prevents data loss
- Always check `empty()` before iterating — handle edge cases
- `switch` is $O(1)$ vs `if-else` $O(n)$ for menu selection

CV Lines for This Project:

- Built a CLI-based inventory management system in C++ supporting product addition, display, and efficient search operations
- Implemented binary search algorithm achieving $O(\log n)$ product lookup by ID using STL vectors and custom product structs
- Designed real-time stock status indicators categorizing products as In Stock, Low Stock, or Out of Stock by quantity
- Integrated file-based data persistence using CSV and `fstream` to retain inventory data across multiple program sessions